Shivam Naik

Orange Cyberdefense Academy Challenge Write Up

Level 0: Internet Protocol Address v4

Problem: Finding the IP address of backup.orangecyberdefense.academy

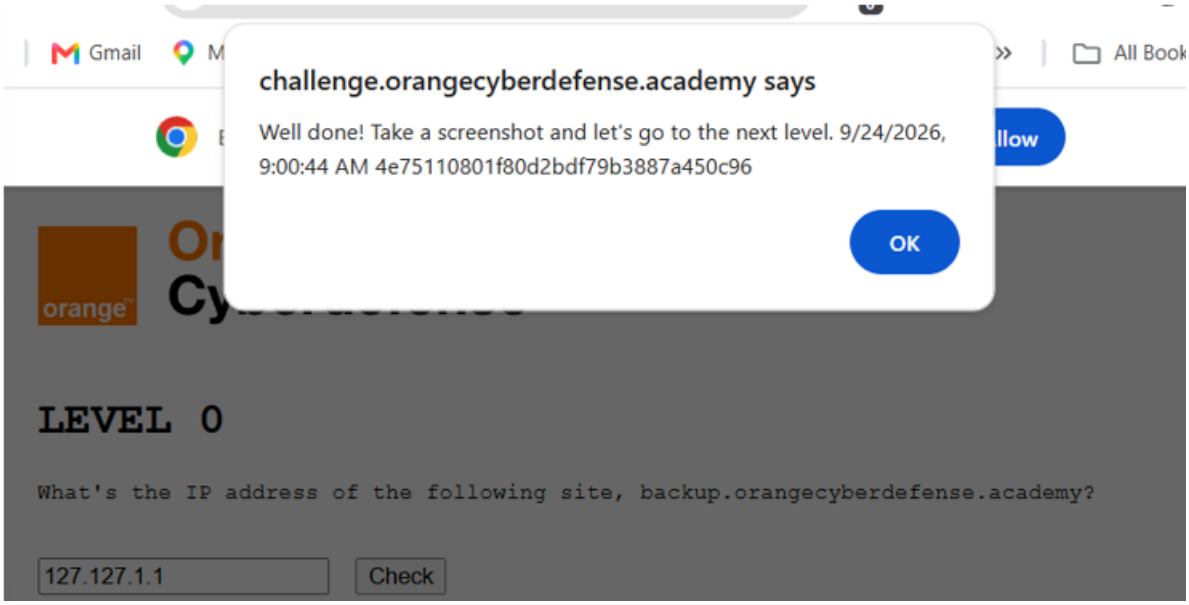
Steps:

- **Tool Selection:** The nslookup utility was selected to query the Domain Name System (DNS) database and resolve the relationship between the target host domain name and its underlying network address. Eg use nslookup www.google.com returns 8.8.8.8 , this is your IP address.
- **Concept Analogy:** A domain name acts as the "phone book" of the internet. While human users navigate using corporate or organizational names, network infrastructure relies on public Internet Protocol (IP) addresses—essentially digital phone numbers—to route communications and allow devices to interact across the World Wide Web.
- **Execution:** Ran the lookup string against the target domain to capture the server's response.

Results with relevant screenshot:

```
tup/
└─(Run: “touch ~/.hushlogin” to hide this message)
┌──(Kaal㉿MAHAKAAL)-[~]
└─$ nslookup backup.orangecyberdefense.academy
Server:         10.255.255.254
Address:        10.255.255.254#53

Non-authoritative answer:
Name:   backup.orangecyberdefense.academy
Address: 127.127.1.1
```



Level 1: Parsing the password in the web page of the challenge

Problem: Finding the hidden password in the login page of the challenge

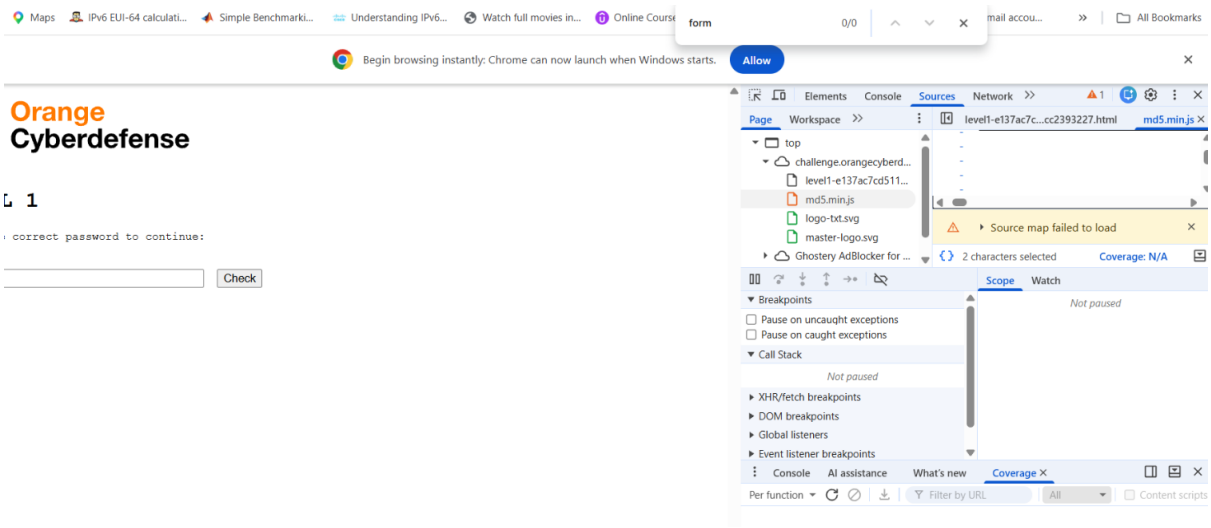
Steps: passwords can be hidden in a web page`s JavaScript file, under HTML comments, under parameters such as source, form, value. To locate the password, I opened the developer tools in my Google Chrome browser

1. Used Fn +F12 -> opened Chrome Developer tools
2. Located the src tab within the Developer tools window, I utilised the Ctrl+F key combination to parse through the source JavaScript files.
3. I then performed analysis of the src window where I found the level1-e137ac7c..cc2393227.html

4. Lines 43 to 46: I located the password within the level1-e137ac7c..html file
5. The password is Frame-Zoo-Fog-Package2-Choose.

Initial attempt: Thought process:

I tried parsing the password using control F and typing in the keywords: password, value, form, input that did not work, then I tried to locate the JavaScript file of the page under the Sources tab within the developer tools to locate a comment pointing to the password. I located this.

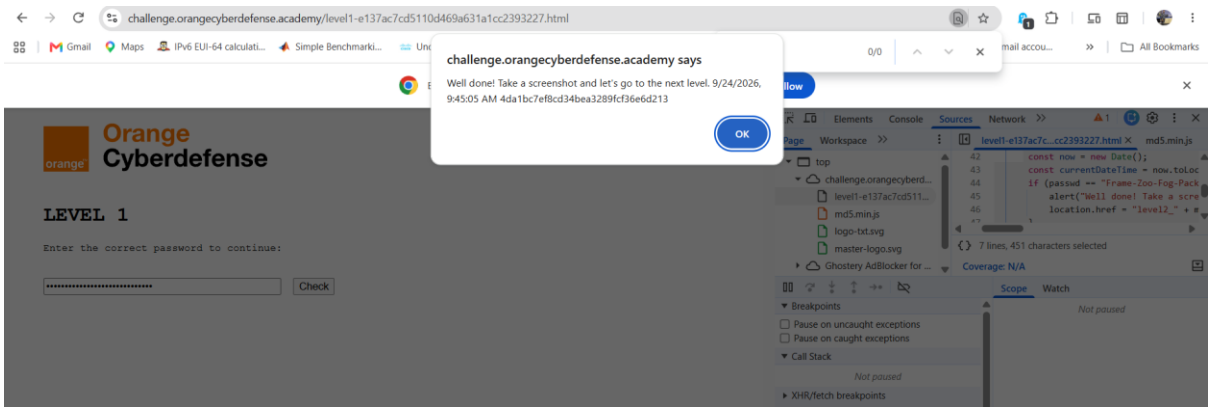


My initial conjecture was to delve md5.mn.js this did not work; I am now parsing in the level1-e137ac7c..cc2393227.html file to see if I can parse the password

After 3 attempts, I decided to inspect the leve1…html file, while scrolling down I located this. I am hinting at Frame-Zoo-Fog-Package2-Choose as the password.



Result: After 3 attempts, my hint has been true the password is Frame-Zoo-Fog-Package2-Choose.



Level 2: Parsing the base key 64 of the hidden secret key

Problem: locating the hidden secret key within a web page under time pressure.

Steps taken:

Thought process:

I spent 20 minutes yesterday, Thursday 24 September 2026 analysing the web page, I first tried Fn +F12 to locate the source JavaScript file embedded in the web page.

What went wrong and how I resolved it?

I failed then went researching on the internet, how to locate the source code on a web page, I came across Ctrl +U, this key combination takes you to the web page JavaScript code, I then located the file level2checkdecoded.js. Based on the below screenshots, I noted that the key is base 64 encoded, I then scripted the base 64 decoding script in Python on Linux to decode the string. The result is base6SD_is_easyflng, this is incorrect. I have fixed the input string within my base64 encoding python script , by feeding it the hash value YmF…Gluz== into my base64 algorithm which then decoded it as base64_is_encoding , hashing is used to ensure integrity of data and base64 is basic obfuscation of important data.

After 18 hours of persistent penetration testing and systems analysis, I have found the secret key which is base64_is_encoding this is the correct key for the level 2 website



What I think may cause this problem?

This error sends me back to level 0, I have been attempting to find the secret key since yesterday Thursday 24 September 2026 10am SAST. The website times out and brings me back to level 0. On a website that normally has ecommerce, this error can chase customers away. What I am experiencing is a form of Denial-of-Service attack.

```
    html { overflow-y: scroll; }
    a, a:visited, a:active { color: #000000; text-decoration: underline; font-size: 15px;}
    a:hover { text-decoration: none }
    body { margin: 0px 0px 0px 0px; font-family: courier new,monospace; font-size: 13px; color: #000000; text-decoration: none; background-
    table { border-collapse: collapse; }
    img { border: 0px; }
    td { padding-left:50px; }
    -->
    </style>
</head>
<body onLoad="setTimeout('delayer()', 5000)">
    <table>
        <tr>
            <td></br></td>
        </tr>
        <tr>
            <td><img alt="" src="master-logo.svg" width='58px'/>  <img alt="" src="logo-txt.svg" width='212px' /></td>
        </tr>
        <tr>
            <td></br></td>
        </tr>
        <tr>
```

1. Transition to Server-Side Session Management (Remediating the 5-Second Timeout)

- **The Vulnerability Identified:** The current application relies on an aggressive client-side JavaScript execution loop embedded directly within the document body: <body onLoad="setTimeout('delayer()', 5000)">.
- **The Developer Fix:**
    - Completely remove the client-side setTimeout() and delayer() window triggers from the HTML markup templates.

- Transition all session state tracking, authentication token tracking, and lifecycle rules to the **server-side application layer** (using secure, HTTP-only session cookies or encrypted JWT structures).
- Enforce standard corporate inactivity timeouts (e.g., a rolling 15 to 30 minutes of absolute idle time) rather than a restrictive 5-second execution window, which severely damages user transaction flows.

2. Implement Strict Server-Side Data Deserialization (Remediating Cryptographic Failures)

- The Vulnerability Identified**:** Authentication validation happens purely on the client-side using the browser's native btoa() binary-to-ascii serialization routine against a static, easily extracted base64 hash.
- The Developer Fix:
  - Never perform authentication parameter validation within client-side code blocks.
  - Enforce all cryptographic parsing, input checking, and data deserialization on the backend server.
  - When handling base64 data pipelines, implement robust catch blocks to gracefully capture formatting anomalies and dynamic character adjustments to prevent server-side crash triggers or unexpected execution state loops.

3. Enforce Strict Input Boundary Controls (Mitigating Memory Corruption & Buffer Overflows)

- The Risk Profile**:** Applications capturing raw data arrays without strict input parameters risk exposing structural endpoints to memory exhaustion or buffer overflow vectors if data allocation thresholds are breached.
- The Developer Fix:
  - Enforce centralized data validation filters at the API gateway layer using secure **parameter binding**.
  - Apply strict maximum length restrictions (max length) and structural whitelist character validation (regex filters) on all form submission pipelines.
  - Leverage memory-safe modern compilation frameworks and automated sanitization libraries to completely isolate array buffers from adjacent system memory structures.

Methodology:

1. Use Ctrl+U to access the web page, the JavaScript file is a .js file containing the website logic under the <src> ..> in the HTML markup
2. I located the level2-checkdecoded.js on line 41

```
<tr>
    <td>
        <!-- content -->
        <script type="text/javascript" src="level2-checkdecoded.js">
        <!--

        //-->
        </script>

        What is the secret key? It's not really that secret:<br /><br /><br /><form actic

        <!-- end content -->
    </td>
```

3. The code that notifies us of the hidden key .The line if(btoa(passwd)==”YmFzZV8..ZGLuZw ==” contains the string required for the secret key

```
function Try(passwd) {
    const now = new Date();
    const currentDateTime = now.toLocaleString();
    if (btoa(passwd) == "YmFzZV82NF9pc19lbmNvZGluZw==") {
        alert("Well done! Take a screenshot and let's go to the next level. "+currentDateTime + " "+
md5(currentDateTime));
        location.href = "level3-" + passwd + ".html";
    }
    else {
        alert("Nope...");
```

The following script shows me my initial decoding, I failed and will alter the script to ensure that it can parse the blue highlighted key in base64 encoding. The new script will take the highlighted key and decode it for us.

```python
import base64

def decode_base64_safe(encoded_text: str) -> str:
    # 1. Clean whitespace or newlines from input
    encoded_text = encoded_text.strip()

    # 2. Add missing '=' padding characters dynamically
    # Base64 string length must be a multiple of 4
    missing_padding = len(encoded_text) % 4
    if missing_padding:
        encoded_text += '=' * (4 - missing_padding)

    try:
        # 3. Decode the clean base64 string
        decoded_bytes = base64.b64decode(encoded_text)
        return decoded_bytes.decode("utf-8", errors="ignore")
    except Exception as e:
        return f"Decoding Error: {str(e)}"

if __name__ == "__main__":
    # Test your target string directly
    target_string = "YmFzZTZTRF9pc19lYXN5ZmxuZw=="

    print("--- Technical Output ---")
    print("Original String:", target_string)
    print("Decoded Flag:   ", decode_base64_safe(target_string))
```

```
Original String: YmFzZTZTRF9pc19lYXN5ZmxuZw==
Decoded Flag:    base6SD_is_easyflng
ahakaalhiv@MAHAKAAL:~/base64_encodings$ nano base64_encode.py
ahakaalhiv@MAHAKAAL:~/base64_encodings$ python3 base64_encode.py
--- Technical Output ---
Original String: YmFzZTZTRF9pc19lYXN5ZmxuZw==
Decoded Flag:    base6SD_is_easyflng
ahakaalhiv@MAHAKAAL:~/base64_encodings$
```

I have found the solution by manipulating this line, I have found the encoded sting base64_is_encoding.

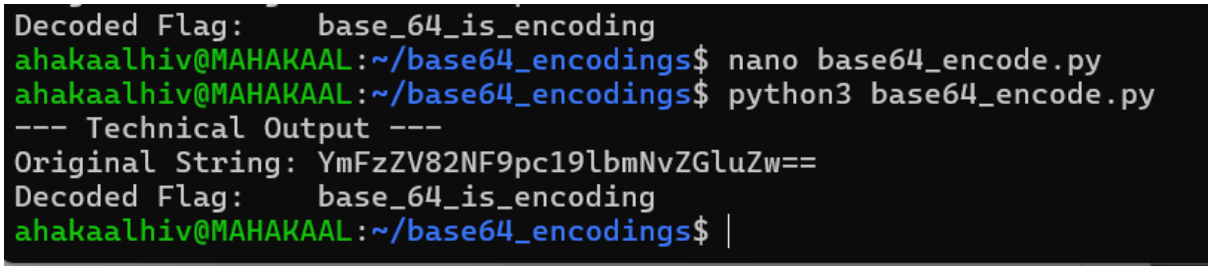
```python
        encoded_text += '=' * (4 - missing_padding)

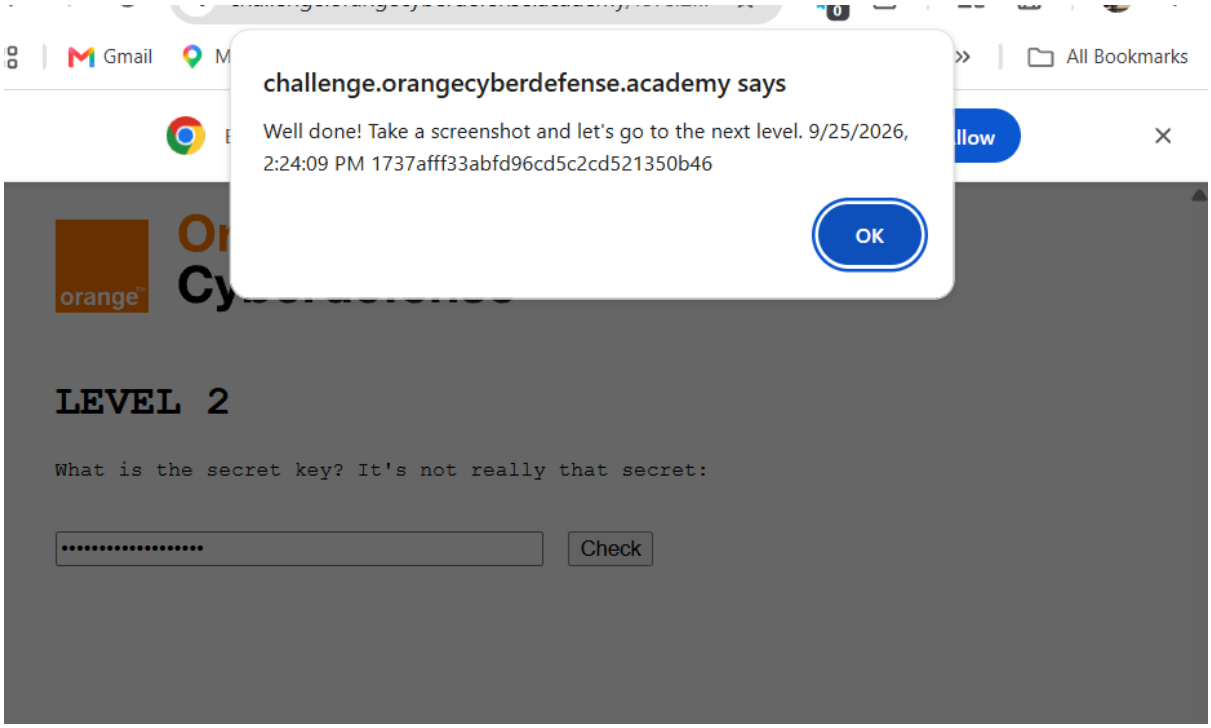
    try:
        # 3. Decode the clean base64 string
        decoded_bytes = base64.b64decode(encoded_text)
        return decoded_bytes.decode("utf-8", errors="ignore")
    except Exception as e:
        return f"Decoding Error: {str(e)}"

if __name__ == "__main__":
    # Test your target string directly original key =YmFzZV82NF9pc19lbmNvZGluZw==
    target_string = "YmFzZV82NF9pc19lbmNvZGluZw=="

    print("--- Technical Output ---")
    print("Original String:", target_string)
    print("Decoded Flag:   ", decode_base64_safe(target_string))
```

The solution





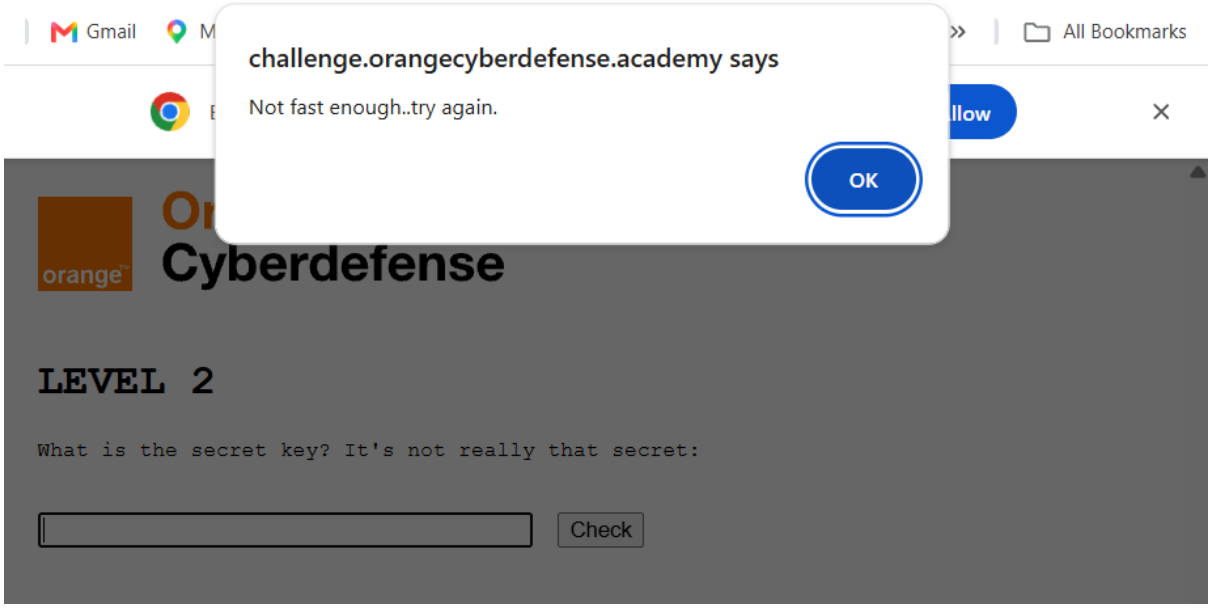
Level 2 has a vulnerability on line 24 (use Ctrl+U to access code of website) , each time you try to move to the next level this error is present. I isolated the cause of the error on line 24, the line reads as follows

```
24    <body onLoad="setTimeout('delayer()', 5000)">
```

The delayer parameter ()..,5000) causes the page to time out and restart the challenge. If the website were an e commerce website, we may have frustrated customers and lose revenue due to this vulnerability, the fix would be removing the line and keep line 24 as <body> this way we have a website that allows optimal performance and keeps our customers happy.

An illustration of how the line should be

```
    </style>
</head>
<body>
    <table>
        <tr>
            <td></br></td>
```



Conclusion & Recommendation:
During the evaluation of Level 2, a client-side timeout variable was identified within the page's source code (line 24, interacting with the <body /> element). When triggered, this script forcefully resets the website back to the landing page, preventing the user from proceeding.

Risk Rating: Low / Informational

Impact:
While this does not directly expose sensitive data, it introduces a significant usability flaw and a client-side Denial of Service (DoS) risk. If deployed on a production environment (such as an e-commerce platform), this mechanism could prematurely interrupt critical user workflows, halt data transit, and disrupt the checkout process—leading to service degradation and customer frustration. Furthermore, because this logic relies strictly on front-end code, a malicious actor can easily manipulate or disable the timeout variable within their local session to bypass intended session limitations.

Remediation Recommendation:

It is recommended that the Orange Cyberdefense development team remove this disruptive front-end timeout variable. Session management and inactivity timeouts should be handled securely on the server-side rather than relying on client-side UI scripts.